

Celebal Summer Internship 2026

Week 2 Task: E-Commerce Sales Database

Submitted by : Shauryansh chauhan (MMDU)

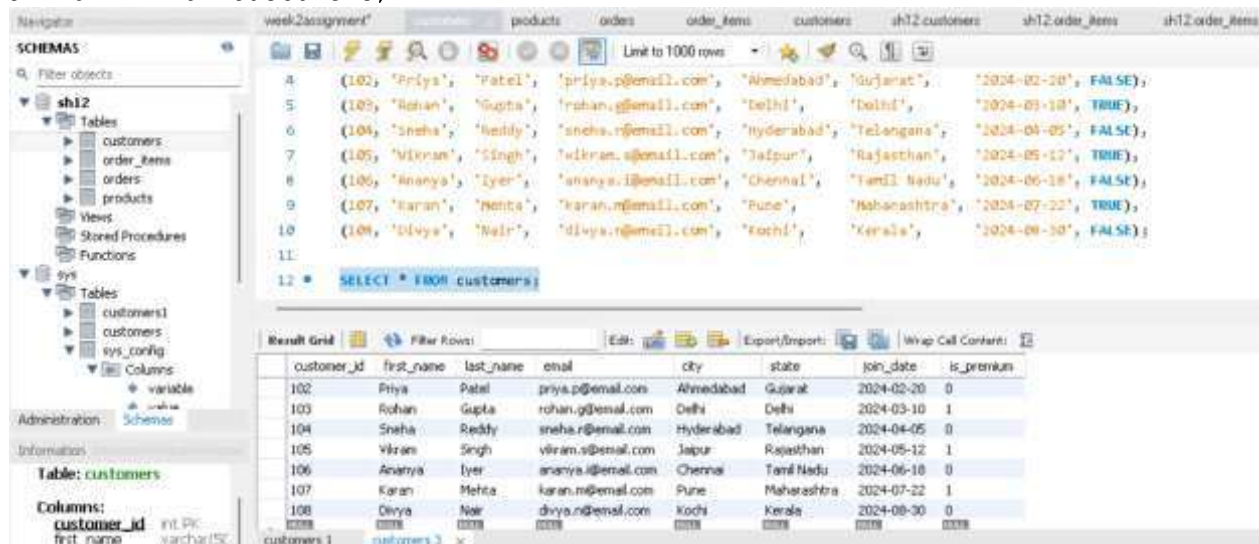
Section A — SQL Basics: Solutions & Explanations

Q1. Write a query to display all columns and rows from the customer's table.

The asterisk (*) symbol is a wildcard character that tells the database to fetch every single column available in the target table.

SQL

```
SELECT * FROM customers;
```



customer_id	first_name	last_name	email	city	state	join_date	is_premium
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.v@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Q2. Retrieve only the first_name, last_name, and city of all customers.

Instead of using *, you explicitly type out the specific column names separated by commas to keep your query efficient and focused.

SQL

```
SELECT first_name, last_name, city
```

FROM customers;

The screenshot shows a database management tool interface. On the left, a 'SCHEMAS' pane shows a tree view with 'sh12' and 'sys' schemas. The 'sh12' schema contains tables: 'customers', 'order_items', 'orders', and 'products'. The 'customers' table is selected, and its columns are listed: 'customer_id', 'first_name', 'last_name', 'email', 'city', 'state', and 'join_date'. The main pane shows a SQL query: `SELECT * FROM customers;` and its results. The results are displayed in a grid with columns: 'first_name', 'last_name', and 'city'. The data rows are: (106, 'Ananya', 'Iyer'), (107, 'Iraan', 'Mehta'), (108, 'Divya', 'Nair'). The bottom pane shows the 'Output' tab with a message: '8 rows returned'.

Q3. List all unique categories available in the products table.

The `DISTINCT` keyword eliminates duplicate rows from your query results, leaving you with a clean list of unique values.

SQL

```
SELECT DISTINCT category
FROM products;
```

The screenshot shows a database management tool interface. On the left, a 'SCHEMAS' pane shows a tree view with 'sh12' and 'sys' schemas. The 'sh12' schema contains tables: 'customers', 'order_items', 'orders', and 'products'. The 'products' table is selected, and its columns are listed: 'product_id', 'category', 'product_name', 'price', 'stock', and 'supplier'. The main pane shows a SQL query: `SELECT DISTINCT category` and its results. The results are displayed in a grid with columns: 'category'. The data rows are: 'Electronics', 'Clothing', and 'Home'. The bottom pane shows the 'Output' tab with a message: '3 rows returned'.

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Looking at the `CREATE TABLE` definitions, the Primary Keys are:

- **customers:** `customer_id`
- **products:** `product_id`
- **orders:** `order_id`

- **order_items:** item_id

: A Primary Key serves as the absolute digital fingerprint for a row. It must be **unique** so the database can distinguish one specific record from another without confusion. It must be **NOT NULL** because an empty or missing identifier means a record cannot be referenced or linked by other tables (Foreign Keys), breaking the relational integrity of the entire database.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

The constraints applied to the email column are **UNIQUE** and **NOT NULL**.

If you try to insert a duplicate email, MySQL will stop the operation, reject the new row, and throw an error message similar to this:

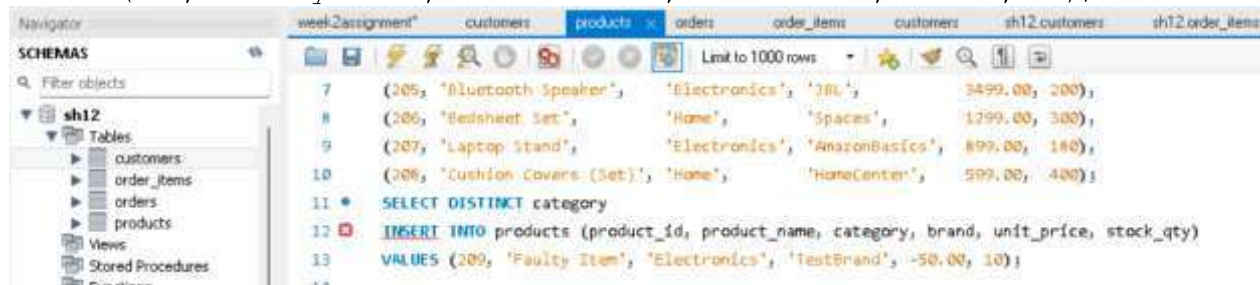
```
Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key
'customers.email'
```

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

The INSERT Statement:

SQL

```
INSERT INTO products (product_id, product_name, category, brand, unit_price,
stock_qty)
VALUES (209, 'Faulty Item', 'Electronics', 'TestBrand', -50.00, 10);
```



The screenshot shows a MySQL database interface with a table named 'products' containing several rows of product data. The table has columns for product_id, product_name, category, brand, unit_price, and stock_qty. The data includes items like 'Bluetooth Speaker', 'Bedsheet Set', 'Laptop Stand', and 'Cushion Covers (Set)'. Below the table, an SQL query is being executed, which is an INSERT statement for a product with a negative unit price. The query is highlighted in red, indicating an error.

product_id	product_name	category	brand	unit_price	stock_qty
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
206	Bedsheet Set	Home	Spaced	1299.00	300
207	Laptop Stand	Electronics	AmazonBasics	899.00	100
208	Cushion Covers (Set)	Home	HomeCenter	599.00	400

```
SELECT DISTINCT category
INSERT INTO products (product_id, product_name, category, brand, unit_price, stock_qty)
VALUES (209, 'Faulty Item', 'Electronics', 'TestBrand', -50.00, 10);
```

MySQL will block the execution and return a Check Constraint Violation error. The specific constraint preventing this is **CHECK (unit_price > 0)** which was declared inside your **CREATE TABLE products** statement to ensure that no item can ever be listed with a zero or negative price.

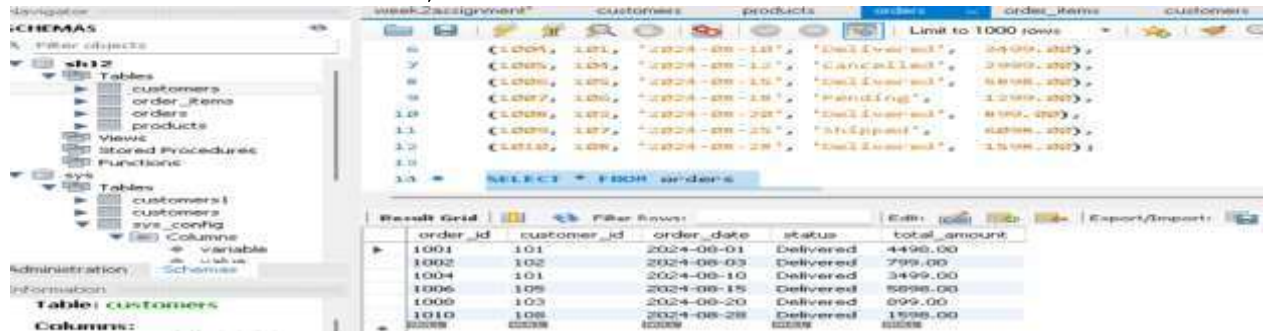
Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

SQL

```
SELECT * FROM orders
```

```
WHERE status = 'Delivered';
```



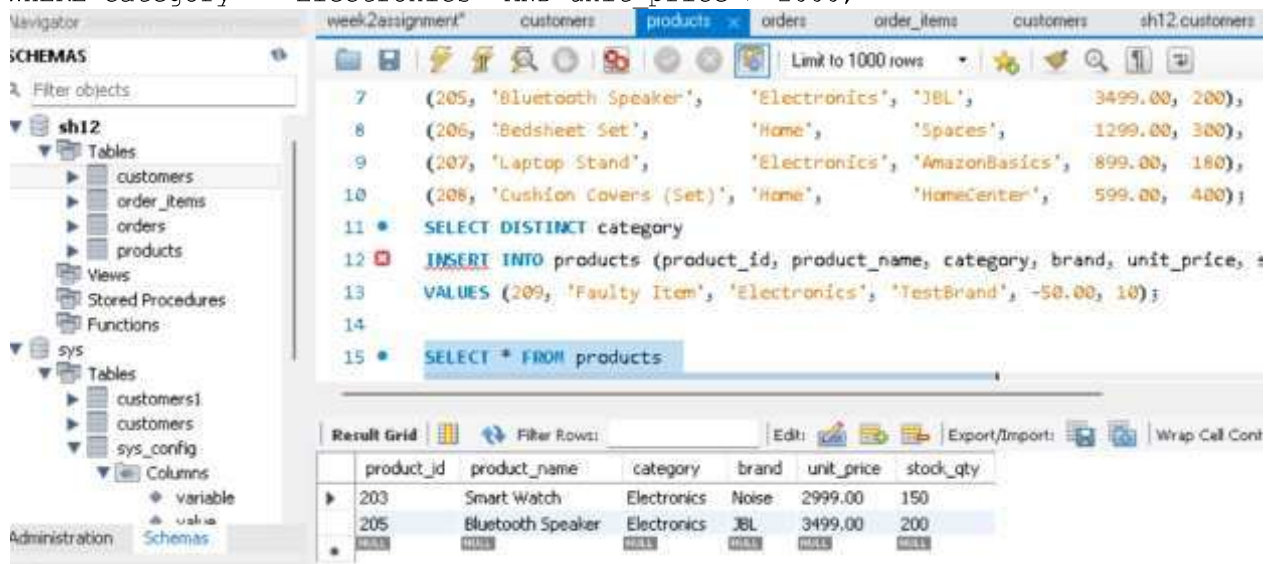
order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4499.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5999.00
1009	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1999.00

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

SQL

```
SELECT * FROM products
```

```
WHERE category = 'Electronics' AND unit_price > 2000;
```



product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

Note: To keep this query index-friendly (SARGable), we use a date range instead of wrapping join_date in a YEAR() function.

SQL

```
SELECT * FROM customers
```

```
WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31'
```

AND state = 'Maharashtra';

The screenshot shows a database management tool interface. On the left, the 'SCHEMAS' pane displays a tree view with 'sh12' and 'sys' databases. The 'sh12' database contains tables 'customers', 'order_items', 'orders', and 'products'. The 'sys' database contains tables 'customers1', 'customers', and 'sys_config'. The main pane shows a SQL query editor with the following code:

```

12 * SELECT * FROM customers;
13
14 * SELECT first_name, last_name, city
15 FROM customers;
16
17 * SELECT * FROM customers
18 WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31'
19 AND state = 'Maharashtra';
20

```

Below the query editor, the 'Result Grid' shows the results of the query. The grid has columns: customer_id, first_name, last_name, email, city, state, join_date, and is_premium. The results are as follows:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

SQL

```

SELECT * FROM orders
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
AND status != 'Cancelled';

```

The screenshot shows a database management tool interface. On the left, the 'SCHEMAS' pane displays a tree view with 'sh12' and 'sys' databases. The 'sh12' database contains tables 'customers', 'order_items', 'orders', and 'products'. The 'sys' database contains tables 'customers1', 'customers', and 'sys_config'. The main pane shows a SQL query editor with the following code:

```

13
14 * SELECT * FROM orders
15 WHERE status = 'Delivered';
16
17 * SELECT * FROM orders
18 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
19 AND status != 'Cancelled';
20

```

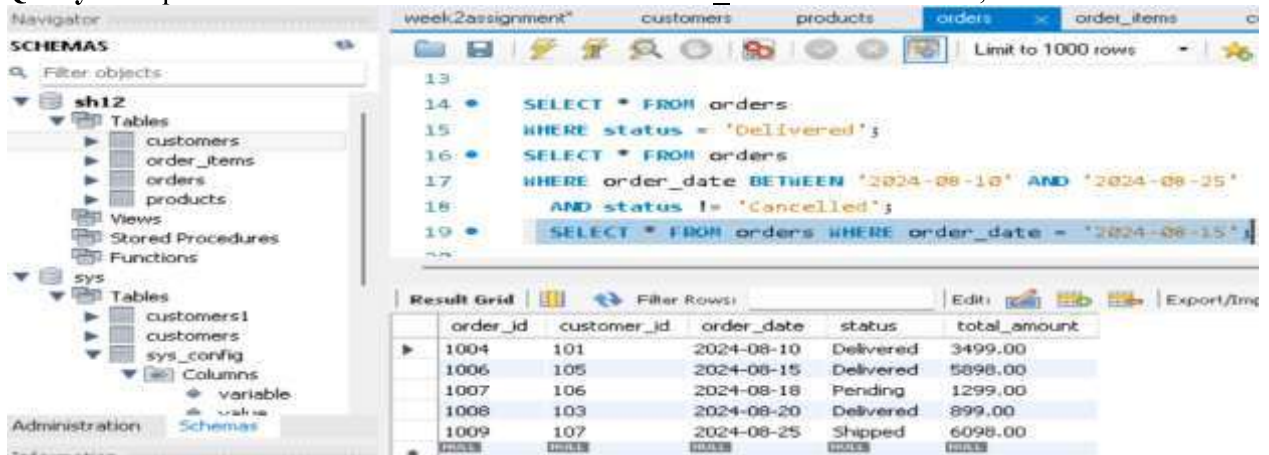
Below the query editor, the 'Result Grid' shows the results of the query. The grid has columns: order_id, customer_id, order_date, status, and total_amount. The results are as follows:

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

- Explanation:** An index creates a sorted, highly optimized data structure (like the index at the back of a book) for the `order_date` column. Instead of scanning every single row in the table to find specific dates (a "full table scan"), the database can jump straight to the exact location of those dates, massively speeding up the query.

- **Query:** ``sql SELECT * FROM orders WHERE order_date = '2024-08-15';

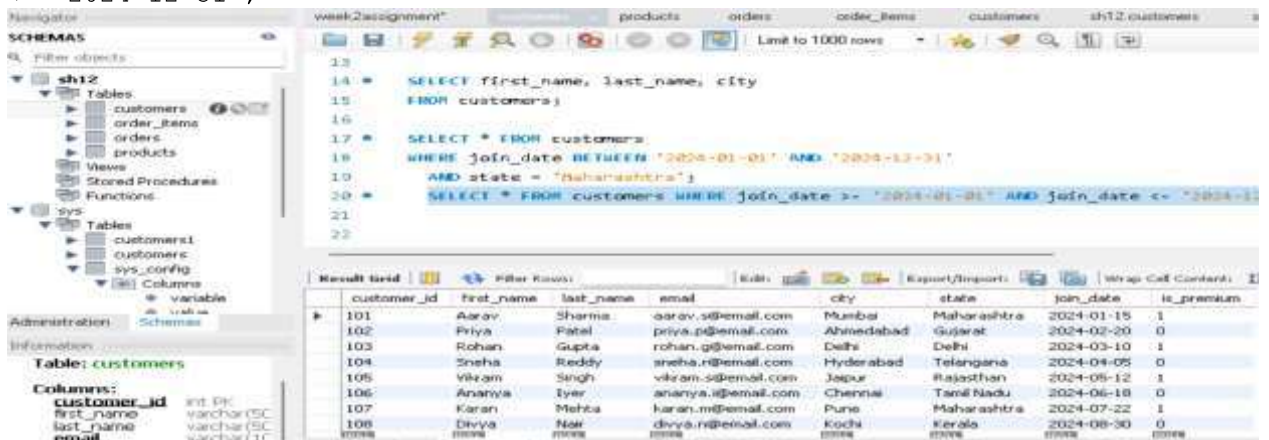


Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

- **Explanation:** No, the index would *not* be used. Because you wrapped the column in the `YEAR()` function, the database is forced to evaluate that function on every single row before it can check the condition. This breaks the index usage (making it non-SARGable) and causes a slow full table scan.
- **Query:**

SQL

`SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date <= '2024-12-31';`

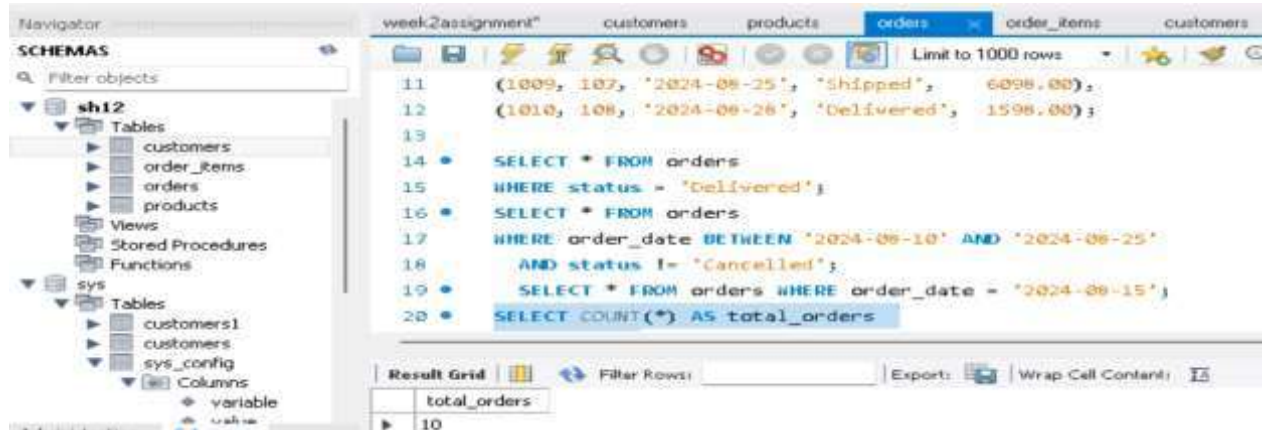


Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

SQL

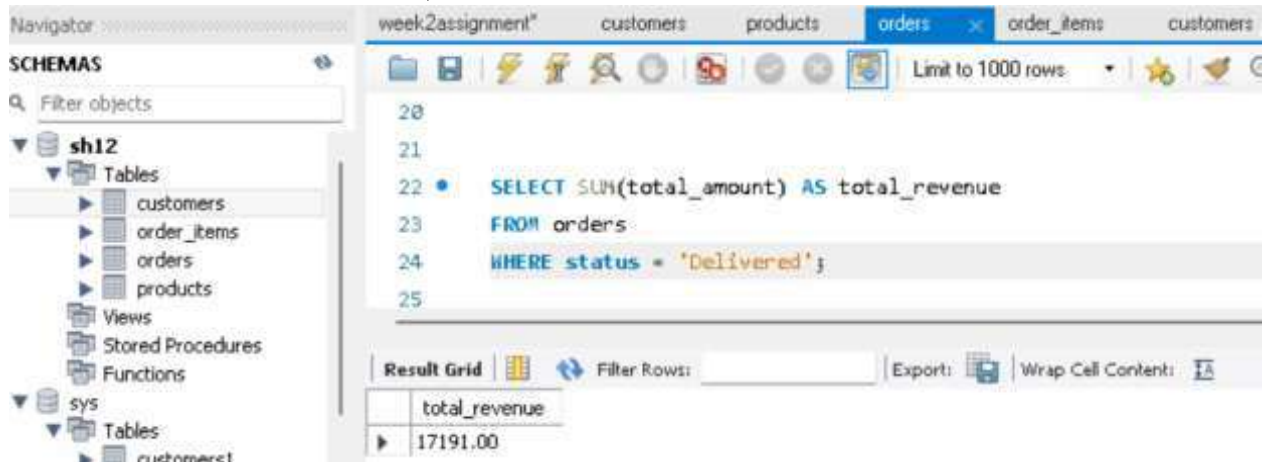
```
SELECT COUNT(*) AS total_orders
FROM orders;
```



Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

SQL

```
SELECT SUM(total_amount) AS total_revenue
FROM orders
WHERE status = 'Delivered';
```



Q15. Calculate the average unit_price of products in each category.

SQL

```
SELECT category, AVG(unit_price) AS avg_price
FROM products
```

GROUP BY category;

The screenshot shows a SQL IDE interface. On the left, a 'SCHEMAS' tree lists databases 'sh12' and 'sys'. The 'sh12' database contains tables 'customers', 'order_items', 'orders', and 'products'. The 'sys' database contains tables 'customers1' and 'customers'. The main editor shows a query for the 'products' table:

```
19 SELECT category, AVG(unit_price) AS avg_price
20 FROM products
21 GROUP BY category
22
23
```

Below the query editor, the 'Result Grid' displays the following data:

category	avg_price
Electronics	2224.000000
Clothing	2699.000000
Home	949.000000

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

SQL

```
SELECT status, COUNT(order_id) AS order_count, SUM(total_amount) AS
total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

The screenshot shows a SQL IDE interface. On the left, a 'SCHEMAS' tree lists databases 'sh12' and 'sys'. The 'sh12' database contains tables 'customers', 'order_items', 'orders', and 'products'. The 'sys' database contains tables 'customers1', 'customers', and 'sys_config'. The main editor shows a query for the 'orders' table:

```
26 SELECT status, COUNT(order_id) AS order_count, SUM(total_amount) AS total_revenue
27 FROM orders
28 GROUP BY status
29 ORDER BY total_revenue DESC
30
31
32
33
34
35
```

Below the query editor, the 'Result Grid' displays the following data:

status	order_count	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

SQL

```
SELECT category, MAX(unit_price) AS most_expensive, MIN(unit_price) AS
cheapest
FROM products
```

GROUP BY category;

The screenshot shows the SQL Developer interface with two queries in the 'products' tab. The first query calculates the average unit price by category. The second query calculates the maximum and minimum unit prices by category. The results are displayed in a grid below the queries.

```
17
18
19 • SELECT category, AVG(unit_price) AS avg_price
20 FROM products
21 GROUP BY category;
22 • SELECT category, MAX(unit_price) AS most_expensive, MIN(unit_price) AS cheapest
23 FROM products
24 GROUP BY category;
25
26
```

category	most_expensive	cheapest
Electronics	3499.00	899.00
Clothing	4599.00	799.00
Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000.

SQL

```
SELECT category
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
```

The screenshot shows the SQL Developer interface with a query in the 'products' tab. The query filters product categories where the average unit price is greater than 2000. The results are displayed in a grid below the query.

```
23 FROM products
24 GROUP BY category;
25
26 • SELECT category,
27 AVG(unit_price)
28 AS avg_price
29 FROM products
30 GROUP BY category
31 HAVING AVG(unit_price) > 2000;
32
33
```

category	avg_price
Electronics	2224.000000
Clothing	2699.000000

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

SQL

```
SELECT o.order_id,  
o.order_date,  
c.first_name,  
c.last_name,  
o.total_amount  
FROM orders o  
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

The screenshot shows a database management tool interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'customers', 'order_items', 'orders', and 'products'. The 'customers' table is selected, and its columns are listed: customer_id (int PK), first_name (varchar(50)), last_name (varchar(50)), email (varchar(100)), city (varchar(50)), state (varchar(50)), and join_date (date). The main editor shows the SQL query for Q19, which is an INNER JOIN between the 'orders' and 'customers' tables. The query is:
`SELECT o.order_id,
o.order_date,
c.first_name,
c.last_name,
o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;`
Below the query editor, the 'Result Grid' displays the results of the query. It shows 10 rows of data, each with columns for order_id, order_date, first_name, last_name, and total_amount. The results are as follows:

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any).

SQL

```
SELECT c.customer_id,
```

```

c.first_name,
c.last_name,
o.order_id,
o.order_date,
o.total_amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;

```

The screenshot shows a database management tool interface. On the left, the 'Navigator' pane displays the 'sh12' database schema with tables: customers, order_items, orders, products, Views, Stored Procedures, and Functions. Below this, the 'sys' database schema is also visible. The main editor pane shows a SQL query with line numbers 19 to 30. The query filters for customers in Maharashtra and retrieves customer details along with their orders. The 'Result Grid' at the bottom displays the query results in a table format.

SQL Query:

```

19 AND state = 'Maharashtra';
20 SELECT * FROM customers WHERE join_date >= '2024-01-
21 SELECT c.customer_id,
22 c.first_name,
23 c.last_name,
24 o.order_id,
25 o.order_date,
26 o.total_amount
27 FROM customers c
28 LEFT JOIN orders o ON c.customer_id = o.customer_id;
29
30

```

Result Grid:

	customer_id	first_name	last_name	order_id	order_date	total_amount
▶	101	Aarav	Sharma	1001	2024-08-01	4498.00
	101	Aarav	Sharma	1004	2024-08-10	3499.00
	102	Priya	Patel	1002	2024-08-03	799.00
	103	Rohan	Gupta	1003	2024-08-05	7498.00
	103	Rohan	Gupta	1008	2024-08-20	899.00
	104	Sneha	Reddy	1005	2024-08-12	2999.00
	105	Vikram	Singh	1006	2024-08-15	5898.00
	106	Ananya	Iyer	1007	2024-08-18	1299.00
	107	Karan	Mehta	1009	2024-08-25	6098.00
	108	Divya	Nair	1010	2024-08-28	1598.00

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

```

SQL
SELECT o.order_id,
p.product_name,
oi.quantity,
oi.unit_price,
oi.discount_pct
FROM orders o
INNER JOIN order_items oi ON o.order_id = oi.order_id

```

```
INNER JOIN products p ON oi.product_id = p.product_id;
```

The screenshot shows a database management tool interface. On the left, a schema tree displays tables: customers, order_items, orders, and products. The main editor shows a SQL query:

```
SELECT o.order_id,
p.product_name,
oi.quantity,
oi.unit_price,
oi.discount_pct
FROM orders o
INNER JOIN order_items oi ON o.order_id = oi.order_id
INNER JOIN products p ON oi.product_id = p.product_id;
```

Below the query, the 'Result Grid' displays the following data:

order_id	product_name	quantity	unit_price	discount_pct
1006	Wireless Earbuds	1	1499.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1005	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1009	Bluetooth Speaker	1	3499.00	0.00
1007	Bedsheet Set	1	1299.00	0.00
1010	Bedsheet Set	1	1299.00	0.00
1001	Laptop Stand	1	899.00	10.00
1008	Laptop Stand	1	899.00	0.00
1009	Cushion Covers (...)	2	599.00	15.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

- **LEFT JOIN** returns all rows from the *left* table (e.g., customers) and any matching rows from the right table (orders). If a customer hasn't ordered, they still show up with NULL for their order details.
- **RIGHT JOIN** is the exact reverse. It returns everything from the *right* table. customers LEFT JOIN orders gives the same logical output as orders RIGHT JOIN customers.
- **FULL OUTER JOIN** returns all records when there is a match in *either* left or right table. You would use this if you wanted a master list of absolutely every customer and absolutely every order, including rogue data like an order missing a customer ID, or a customer with zero orders.

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

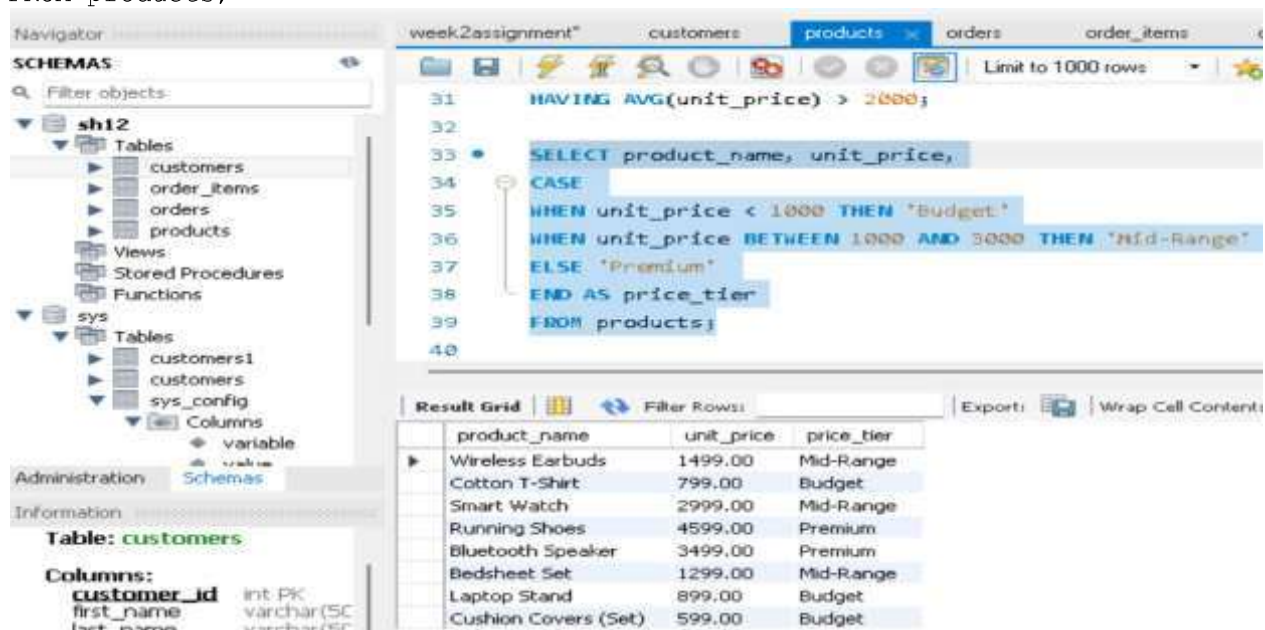
- **Foreign Keys:**
 1. orders.customer_id references customers.customer_id
 2. order_items.order_id references orders.order_id
 3. order_items.product_id references products.product_id
- **Explanation:** The database enforces **Referential Integrity**. If you insert customer_id = 999, MySQL will instantly block the transaction and throw a Foreign Key Constraint Violation error because it cannot link the order to a valid parent record in the customers table.

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers.

SQL

```
SELECT product_name, unit_price,  
CASE  
WHEN unit_price < 1000 THEN 'Budget'  
WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'  
ELSE 'Premium'  
END AS price_tier  
FROM products;
```

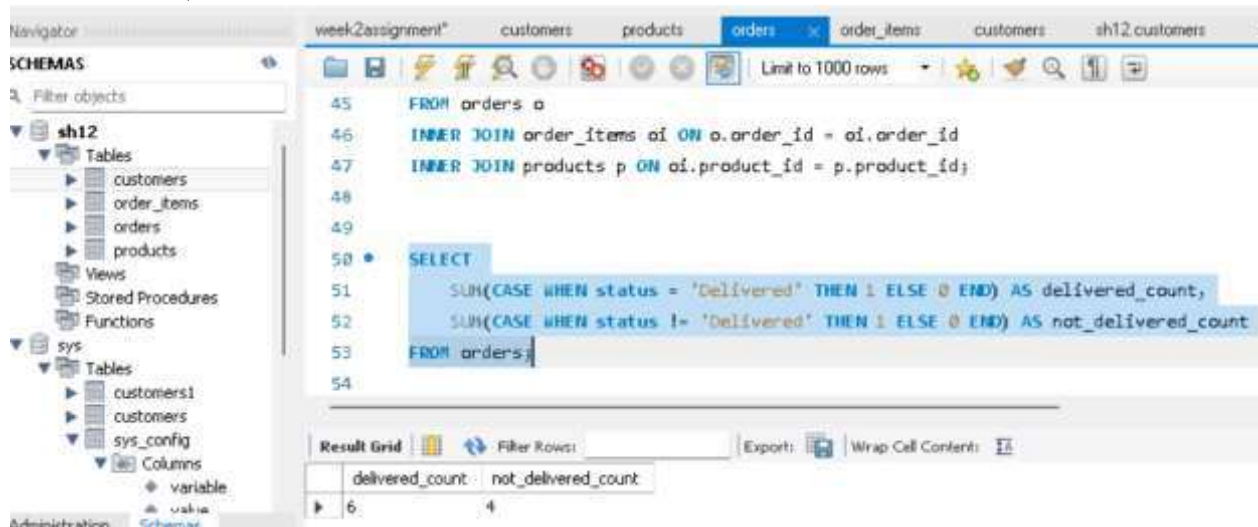


Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered'.

SQL

```
SELECT  
    SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_count,  
    SUM(CASE WHEN status != 'Delivered' THEN 1 ELSE 0 END) AS  
not_delivered_count
```

FROM orders;



Q26. Explain each letter of ACID with a real-world example (e.g., bank transfer).

- **A – Atomicity:** "All or nothing." If you transfer ₹500, your account is debited, and the other is credited. If the credit fails, the debit is completely undone. The transaction cannot be partially completed.
- **C – Consistency:** The database must remain in a valid state following all rules/constraints. For example, if an account balance has a constraint that it cannot drop below ₹0, a transfer that would drop the balance to -₹100 will fail.
- **I – Isolation:** Transactions happening at the exact same time do not interfere with each other. If two people try to withdraw ₹500 from an account with only ₹500 total at the exact same millisecond, the database handles them sequentially so they don't both succeed.
- **D – Durability:** Once a transaction is successfully completed (Committed), it is saved permanently to the physical disk. If the server loses power two seconds later, the bank transfer is still recorded when it turns back on.

Bank Transfer Example

Suppose Alice transfers ₹1,000 to Bob.

1. Atomicity (All or Nothing)

Meaning: A transaction is completed fully or not at all.

Example:

When Alice transfers ₹1,000, the bank must:

1. Deduct ₹1,000 from Alice's account.
2. Add ₹1,000 to Bob's account.

If the system crashes after deducting the money but before adding it to Bob's account, the whole transaction is cancelled.

Importance: Prevents loss of money due to incomplete transactions.

2. Consistency (Data Remains Correct)

Meaning: The database must always remain accurate and follow all rules.

Example:

Before transfer:

- Alice = ₹10,000
- Bob = ₹5,000

After transfer:

- Alice = ₹9,000
- Bob = ₹6,000

The total money remains the same.

Importance: Keeps data correct and reliable.

3. Isolation (Transactions Don't Affect Each Other)

Meaning: Multiple transactions can happen at the same time without causing errors.

Example:

If Alice makes two transfers at the same time, the database processes them properly so the balance does not become incorrect.

Importance: Prevents confusion and wrong account balances.

4. Durability (Data Is Saved Permanently)

Meaning: Once a transaction is completed, it is permanently stored.

Example:

After Alice transfers ₹1,000 and receives a success message, the bank's server crashes. The transfer is still saved and will not be lost.

Importance: Ensures completed transactions remain safe even after system failures.

Q27. Write a SQL transaction that does the following atomically...

SQL

START TRANSACTION;

1. Insert a new order

```
INSERT INTO orders (order_id, customer_id, order_date, status, total_amount)
VALUES (1011, 102, CURDATE(), 'Pending', 1598.00);
```

2. Insert two order items for that order (Using product 206 and 208 for this example)

```
INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price,
discount_pct)
VALUES (5016, 1011, 206, 1, 1299.00, 0),
       (5017, 1011, 208, 1, 599.00, 15);
```

3. Update the stock_qty of the purchased products

UPDATE products

```
SET stock_qty = stock_qty - 1
WHERE product_id IN (206, 208);
```

4. COMMIT the transaction (If testing manually, you highlight and run this .

If an error occurred in steps 1-3, you would highlight and run ROLLBACK; instead).

COMMIT;

The screenshot shows a SQL IDE interface with a script editor and an output window. The script editor contains the following SQL code:

```
28 LEFT JOIN orders o ON c.customer_id = o.customer_id;
29
30 * START TRANSACTION;
31 * INSERT INTO orders (order_id, customer_id, order_date, status, total_amount)
32 * VALUES (1011, 102, CURDATE(), 'Pending', 1598.00);
33 * INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct)
34 * VALUES (5016, 1011, 206, 1, 1299.00, 0),
35 * (5017, 1011, 208, 1, 599.00, 15);
36 *
37 * UPDATE products
38 * SET stock_qty = stock_qty - 1
39 * WHERE product_id IN (206, 208);
40
```

The output window shows the execution results of the script:

#	Time	Action	Message	Duration / Rows
86	17:05:46	SELECT product_name, unit_price, CASE WHEN unit_price > 1000 THEN...	8 row(s) returned	0.000 sec / 0.000 sec
87	17:07:57	SELECT SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS del...	1 row(s) returned	0.000 sec / 0.000 sec
88	17:20:08	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id IN (206, 208)	2 row(s) affected Rows matched: 2 Changed: 2 Warnings: 0	0.063 sec
89	17:20:13	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id IN (206, 208)	2 row(s) affected Rows matched: 2 Changed: 2 Warnings: 0	0.000 sec
90	17:20:48	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id IN (206, 208)	2 row(s) affected Rows matched: 2 Changed: 2 Warnings: 0	0.000 sec
91	17:21:24	START TRANSACTION	0 row(s) affected	0.016 sec